Môn học: Đồ họa máy tính · Trường Đại học Quốc tế, ĐHQG-HCM

Báo cáo đồ án môn học: Splat Editor

Splat Editor: trình xem và chỉnh sửa cảnh 3D Gaussian Splatting

Lê Anh Kiệt[a]

[a] *Trường Đại học Quốc tế, Đại học Quốc gia TP. Hồ Chí Minh, Việt Nam* MSSV: ITCSIU24047

THÔNG TIN

Sinh viên: Lê Anh Kiệt

Từ khóa:
3D Gaussian Splatting
OpenGL, GLSL
Camera và phép chiếu
Ánh sáng và bóng đổ
Mô phỏng vật lý
Giao diện ImGui

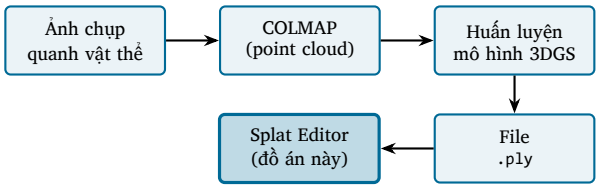
TÓM TẮT

Đồ án xây dựng Splat Editor, một chương trình xem và chỉnh sửa cảnh 3D Gaussian Splatting (3DGS) bằng Python và OpenGL 3.3. Chương trình tự cài đặt pipeline hiển thị 3DGS: đọc file PLY, chiếu ellipsoid thành ellipse, sắp xếp theo độ sâu và trộn alpha. Trên nền đó, đồ án dùng sáu nhóm kỹ thuật của môn học: phép biến đổi, camera, texture, ánh sáng, hoạt họa và tương tác. Người dùng có thể chọn, xóa, hoàn tác và lưu lại file.

1. Giới thiệu và lý thuyết gaussian

1.1. Giới thiệu

3D Gaussian Splatting (3DGS) hiển thị một cảnh 3D bằng hàng triệu ellipsoid mềm, gọi là gaussian [2]. Cảnh được huấn luyện từ ảnh chụp quanh vật thể (Hình 1). File `.ply` thu được luôn có các gaussian lơ lửng. Vì vậy cần một chương trình để xem và làm sạch file. SuperSplat [3] làm việc này trên trình duyệt. Tôi viết một trình chỉnh sửa tương tự bằng Python và OpenGL 3.3.



Hình 1. Quy trình từ ảnh chụp đến Splat Editor. COLMAP [1] tìm vị trí các camera.

Đồ án có hai phần. Phần một là pipeline hiển thị 3DGS, từ file đến shader. Phần hai là sáu nhóm kỹ thuật của môn học (Bảng 1).

Bảng 1. Các nhóm kỹ thuật và các mục.

Nhóm	Nội dung chính	Mục
T1	layer, ma trận **M**, bộ xương nhân vật	2
T2	ma trận nhìn/chiếu, EWA, trộn alpha, camera	3
T3	texture skin của nhân vật	4
T4	màu SH, Phong, shadow map	5
T5	ngày–đêm, va chạm, hoạt họa nhân vật	6
T6	cọ chọn, xóa/hoàn tác, ImGui, HUD	7

1.2. Cấu trúc chương trình

Mã nguồn chia theo vai trò: `core` chỉ chứa dữ liệu và toán nên không phụ thuộc OpenGL, `render` chứa mọi lệnh OpenGL và shader, `ui` lo giao diện, còn `app.py` nối các phần lại. Hình 2 chỉ liệt kê các file đại diện.

```
SplatOpenGL/
|-- main.py              # điểm vào: python main.py [scene.ply]
|-- requirements.txt     # numpy, PyOpenGL, glfw, imgui-bundle
|-- report/              # báo cáo LaTeX
`-- splat_editor/
    |-- app.py           # vòng lặp chính, nối các phần lại
    |-- core/            # dữ liệu và toán, không dùng OpenGL
    |   |-- splats.py    # các mảng gaussian
    |   |-- layer.py     # một file PLY + biến đổi + vùng chọn
    |   |-- scene.py     # tập các layer
    |   |-- math3d.py    # ma trận V, P, góc Euler, quaternion
    |   |-- lighting.py  # mặt trời, đèn, ngày đêm
    |   |-- physics.py   # va chạm voxel, nhảy
    |   |-- avatar_pose.py  # góc khớp nhân vật
    |   |-- lod.py       # chọn gaussian theo mức chi tiết
    |   `-- history.py   # hoàn tác (undo)
    |-- io/              # đọc / ghi file
    |   |-- ply.py       # đọc / ghi PLY
    |   `-- codec.py     # cột PLY <-> Splats
    |-- editor/          # chỉnh sửa
    |   |-- editor.py    # mở, lưu, xoá, chọn
    |   `-- brush.py     # cọ chọn gaussian
    |-- render/          # OpenGL
    |   |-- splat_renderer.py  # vẽ gaussian (2 pass)
    |   |-- sorter.py    # sắp xếp độ sâu (luồng nền)
    |   |-- camera.py    # camera quỹ đạo / bay
    |   |-- avatar_renderer.py # vẽ nhân vật
    |   |-- shader.py    # biên dịch GLSL
    |   |-- shaders/     # splat_prep.vert, splat.vert/.frag, ...
    |   `-- textures/    # skinzom.png
    `-- ui/              # giao diện ImGui
        |-- window.py    # cửa sổ GLFW
        |-- input.py     # bàn phím, chuột
        `-- sections.py  # các bảng điều khiển
```

Hình 2. Cây thư mục của chương trình (chỉ các file chính).

1.3. Dữ liệu đầu vào

Đầu vào là một file `.ply` do quá trình huấn luyện 3DGS tạo ra. Huấn luyện bắt đầu từ point cloud của COLMAP và đặt một gaussian tại mỗi điểm. Sau đó nó lặp lại ba bước. Nó vẽ các gaussian. Nó so ảnh vẽ ra với ảnh chụp thật. Nó cập nhật tham số của từng gaussian. Nó cũng xóa các gaussian vô ích và tách các gaussian quá lớn.

File có hai phần: header dạng văn bản và phần thân nhị phân. Header liệt kê các thuộc tính. Phần thân có một bản ghi cho mỗi gaussian. Mỗi giá trị là một số thực 32 bit. Bảng 2 liệt kê các thuộc tính. Với SH bậc 3, một bản ghi có 62 giá trị, tức 248 byte. Hình 3 là các dòng đầu của một file, xuất ra CSV.

Bảng 2. Các thuộc tính của một gaussian trong file PLY.

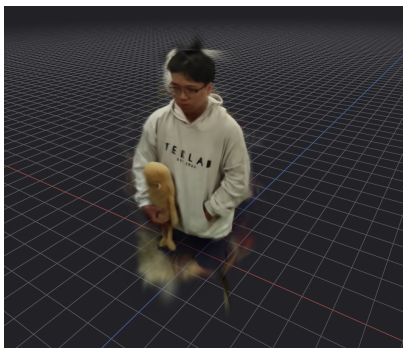
Thuộc tính	Số	Ý nghĩa
`x, y, z`	3	tâm $\boldsymbol{\mu}$
`nx, ny, nz`	3	pháp tuyến, luôn bằng 0, không dùng
`f_dc_0..2`	3	màu cơ bản (SH bậc 0)
`f_rest_*`	45	hệ số SH bậc 1–3
`opacity`	1	độ đục thô
`scale_0..2`	3	logarit của kích thước
`rot_0..3`	4	quaternion quay

```
x,y,z,nx,ny,nz,f_dc_0,                                  f_dc_1,f_dc_2,
2.98395634,0.64003098,1.57549512,0,0,0,0.0184020139,-0.12875317,-0.452952206,
4.22914267,-0.663039207,-0.523675323,0,0,0,-0.150325835,-0.135893673,-0.325479239,
1.13421345,0.410973638,3.93819642,0,0,0,0.346005678,0.0445601828,-0.405238956,
0.604746521,2.51691461,-0.371717632,0,0,0,-0.157206044,-0.176818103,-0.256703675,
0.642261147,-1.39632988,-4.3651309,0,0,0,-0.227364123,-0.334438473,-0.433887929,
-4.2221036,-0.695849061,0.247175366,0,0,0,0.412634581,0.405996054,0.111131459,
```

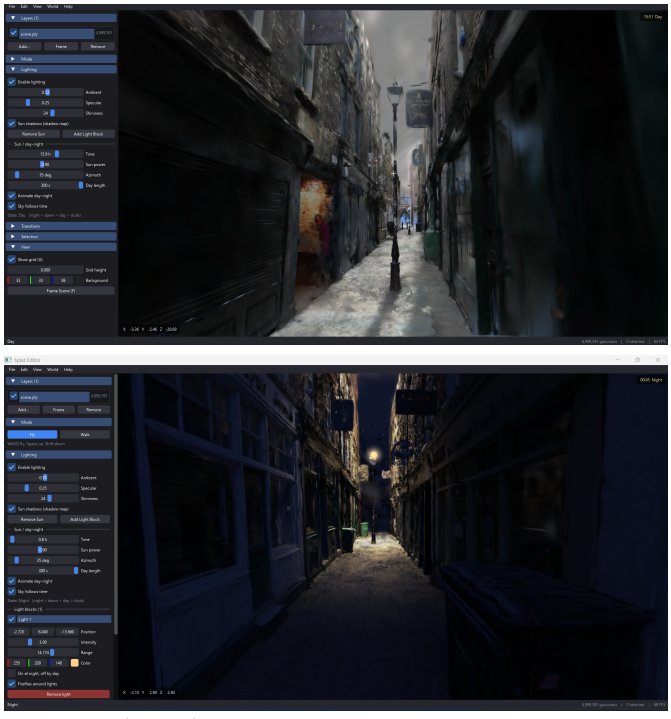
```
opacity,scale_0,scale_1,scale_2,rot_0,rot_1,rot_2,rot_3
-0.0695948228,-0.0550651364,-0.0382503457,-0.0237467252,0.0131118316,0.0629977509,-0.0449319854,-0.
0.000325970963,-0.0160180721,0.00357675459,0.0473092273,0.0389139801,-0.0540050268,-0.0265164431,-0
0.0612790994,0.0957935154,-0.0631214827,0.0422011353,0.0244775601,0.0491610095,0.0470059477,0.07206
-6.13165855,-2.71730924,-2.80323267,-2.86005092,0.876973867,0.112431385,0.0801429227,-0.0182835758
-5.53735924,-2.7295742,-2.76182842,-2.74456096,0.885511041,0.101086691,-0.0186818913,-0.0949812829
-0.0379554592,0.00524016004,0.0189723969,-0.0380355604,-0.0966132209,-0.00303976983,-0.0181281436,-
```

Hình 3. Sáu gaussian đầu tiên của một file PLY, hiển thị dạng CSV. Trên: vị trí, pháp tuyến và màu cơ bản. Dưới: độ đục, kích thước và phép quay. Các cột `f_rest` ở giữa được ẩn đi.

Chương trình ánh xạ file nhị phân thẳng vào bộ nhớ. Vì vậy file có hàng triệu gaussian mở gần như tức thì. Hình 4 là một bản chụp nhỏ của một người. Các đốm mờ quanh người là gaussian lơ lửng mà trình chỉnh sửa phải xóa. Cảnh thử nghiệm chính là một con phố có khoảng 5 triệu gaussian (Hình 5).



Hình 4. File PLY của một người được chụp lại. Các gaussian lơ lửng hiện rõ quanh người và gần mặt đất.



Hình 5. Một cảnh khoảng 5 triệu gaussian trong Splat Editor, ban ngày (trên) và ban đêm (dưới).

1.4. Gaussian và file PLY

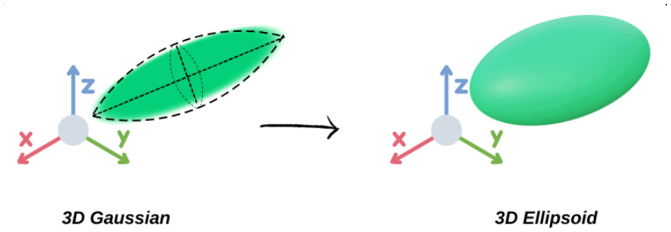
Mỗi gaussian có vị trí $\boldsymbol{\mu}$, màu $\mathbf{c}$, độ đục α, kích thước $\mathbf{s}$ và phép quay $\mathbf{q}$ (quaternion). Mật độ của nó giảm dần từ tâm ra ngoài. Ta có thể hình dung nó là một ellipsoid (Hình 6). File PLY lưu giá trị thô. Chương trình giải mã khi đọc:

- Kích thước: $s_i = \exp(\texttt{scale_i})$.
- Độ đục: $\alpha = 1/(1+e^{-\texttt{opacity}})$.
- Màu: $\mathrm{RGB} = 0{,}5 + C_0\,\texttt{f_dc}$, với $C_0 \approx 0{,}2821$.
- Phép quay: chuẩn hóa quaternion `rot_0..3` về độ dài 1.

Khi lưu file, chương trình làm ngược lại.

Mã 1. `io/codec.py`, `SplatCodec.decode`: giải mã giá trị thô của file PLY

```
SH_C0 = 0.28209479177387814
colors = 0.5 + SH_C0 * np.stack([col('f_dc_0'), col('f_dc_1'), col('
↪ f_dc_2')], axis=1)
alphas = 1.0 / (1.0 + np.exp(-col('opacity')))
scales = np.exp(np.stack([col('scale_0'), col('scale_1'), col('
↪ scale_2')], axis=1))
q = np.stack([col('rot_0'), col('rot_1'), col('rot_2'), col('rot_3')],
↪  axis=1)
length = np.linalg.norm(q, axis=1, keepdims=True)
q = q / np.maximum(length, 1e-12)
```



Hình 6. Gaussian 3D là một đốm mềm. Mặt của nó tại một độ lệch chuẩn là một ellipsoid.

Hình dạng của gaussian nằm trong ma trận hiệp phương sai $\boldsymbol{\Sigma}$. Nó được tạo từ ma trận co giãn $\mathbf{S} = \mathrm{diag}(\mathbf{s})$ và ma trận quay $\mathbf{R}(\mathbf{q})$:

$$\boldsymbol{\Sigma} = \mathbf{R}\,\mathbf{S}\mathbf{S}^{\top}\mathbf{R}^{\top}. \tag{1}$$

Hình 7 cho hai ma trận này. Hình 8 vẽ kết quả cho gaussian đầu tiên của Hình 3. Ba kích thước s_0, s_1, s_2 là các bán trục của ellipsoid. Quaternion xoay các trục này trong không gian. Mục 3.2 chiếu ellipsoid này lên màn hình.

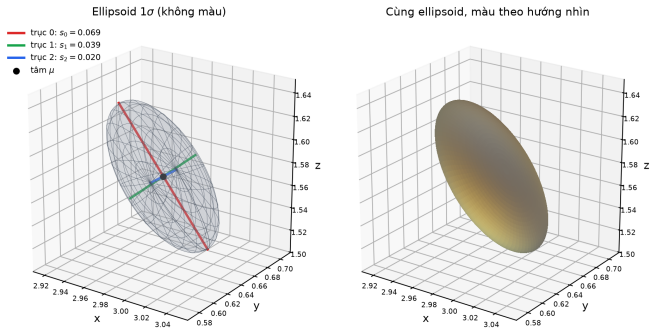
Mã 2. `render/shaders/splat_prep.vert`: dựng $\Sigma = \mathbf{R}\mathbf{S}\mathbf{S}^{\top}\mathbf{R}^{\top}$

```
vec3 scale = vec3(unpackHalf(t1.y), halfToFloat(t1.z & 65535u));
vec4 q = normalize((vec4(uvec4(t1.w, t1.w >> 8u, t1.w >> 16u, t1.w >>
↪  24u)
                         & 255u) - 128.0) / 127.0);
float w = q.x, x = q.y, y = q.z, z = q.w;
mat3 R = mat3(1.0 - 2.0 * (y * y + z * z), 2.0 * (x * y + w * z), 2.0
↪  * (x * z - w * y),
              2.0 * (x * y - w * z), 1.0 - 2.0 * (x * x + z * z), 2.0
↪  * (y * z + w * x),
              2.0 * (x * z + w * y), 2.0 * (y * z - w * x), 1.0 - 2.0
↪  * (x * x + y * y));
mat3 M = R * mat3(scale.x, 0.0, 0.0, 0.0, scale.y, 0.0, 0.0, 0.0,
↪ scale.z);
mat3 sigma = M * transpose(M);
```

$$R = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1-2(q_x^2+q_y^2) \end{bmatrix}$$

$$S = \begin{bmatrix} s_0 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_2 \end{bmatrix}$$

Hình 7. Ma trận quay $\mathbf{R}$ từ quaternion (q_w, q_x, q_y, q_z) và ma trận co giãn $\mathbf{S}$.



Hình 8. Một gaussian trong file. Trái: ellipsoid 1σ với ba trục và tâm $\boldsymbol{\mu}$. Phải: cùng ellipsoid đó với màu phụ thuộc hướng nhìn.

2. T1: Phép biến đổi và mô hình phân cấp

Chương này có ba phần. Hai phần đầu di chuyển cả một layer, phần cuối di chuyển một mô hình phân cấp.

- **Phần 1, biến đổi layer.** Đưa gaussian của một file `.ply` vào thế giới.
- **Phần 2, quay quanh một điểm.** Quay layer quanh một tâm tùy chọn.
- **Phần 3, bộ xương nhân vật.** Dựng ma trận của nhân vật từ một cây khớp.

2.1. Phần 1: Biến đổi layer

Mỗi file `.ply` là một layer. Layer có vector tịnh tiến $\mathbf{t}$, ba góc Euler và hệ số co giãn s. Chúng tạo thành ma trận $\mathbf{M}$. Ma trận này đưa gaussian từ không gian file sang thế giới:

$$\mathbf{p}_w = s\mathbf{A}\mathbf{p}_f + \mathbf{t}, \qquad \boldsymbol{\Sigma}_w = s^2\mathbf{A}\,\boldsymbol{\Sigma}_f\,\mathbf{A}^\top. \tag{2}$$

Ở đây $\mathbf{A} = \mathbf{R}_z\mathbf{R}_y\mathbf{R}_x\mathbf{F}$. $\mathbf{F} = \mathrm{diag}(-1,-1,1)$ là phép lật tùy chọn cho file COLMAP có trục Y hướng xuống. Dữ liệu gaussian không đổi. Chỉ $\mathbf{M}$ thay đổi. Vì vậy di chuyển, xoay hoặc co giãn một layer diễn ra tức thì. Khi lưu, chương trình ghi phép biến đổi vào từng gaussian. Nó cũng xoay các hệ số SH để màu theo hướng nhìn vẫn đúng.

Mã 3. `core/math3d.py` và `core/layer.py`: ma trận $\mathbf{M}$ của layer

```
rx = np.array([[1, 0, 0], [0, cx, -sx], [0, sx, cx]])
ry = np.array([[cy, 0, sy], [0, 1, 0], [-sy, 0, cy]])
rz = np.array([[cz, -sz, 0], [sz, cz, 0], [0, 0, 1]])
return rz @ ry @ rx

rot = self.transform.rotation_matrix() @ (FLIP if flip else np.
↪ identity(3))
m = np.identity(4)
m[:3, :3] = s * rot
m[:3, 3] = t
```

2.2. Phần 2: Quay quanh một điểm

Layer có thể quay quanh điểm $\mathbf{c}$ bằng ma trận $\boldsymbol{\Delta}$:

$$\mathbf{R}' = \boldsymbol{\Delta}\mathbf{R}, \qquad \mathbf{t}' = \boldsymbol{\Delta}(\mathbf{t} - \mathbf{c}) + \mathbf{c}. \tag{3}$$

Mã 4. `core/transform.py`, `Transform.rotate_about`

```
def rotate_about(self, delta, pivot):
    pivot = np.asarray(pivot, float)
    self.rotation = matrix_to_euler(delta @ self.rotation_matrix())
    self.position = list(delta @ (self.translation() - pivot) + pivot
↪ )
```

2.3. Phần 3: Bộ xương nhân vật

Nhân vật hiện ra khi bấm `F5`. Nó có sáu hộp: đầu, thân, hai tay và hai chân. Mỗi hộp có một ma trận riêng. Ma trận này là tích các ma trận khớp từ gốc xuống (Hình 10). Mỗi khớp quay quanh một điểm $\mathbf{p}$:

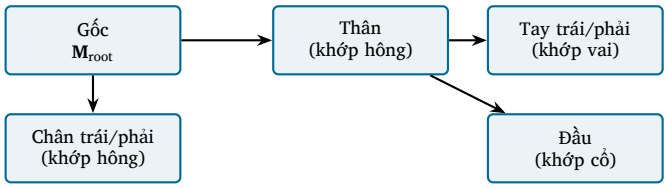
$$\mathbf{M}_{\text{con}} = \mathbf{M}_{\text{cha}}\,\mathbf{T}(\mathbf{p})\,\mathbf{R}\,\mathbf{T}(-\mathbf{p}). \tag{4}$$

Chân là con của gốc. Tay và đầu là con của thân, nên chúng nghiêng theo thân. Các góc khớp đến từ hoạt họa (Mục 6).

Mã 5. `core/avatar_pose.py`, `limb_matrices`: ma trận khớp

```
def leg(angle):
    return (root @ _translate(0, HIP_Y, 0) @ _rot("x", angle)
            @ _translate(0, -HIP_Y, 0))

def arm(px, angles):
    return (upper @ _translate(px, SHOULDER_Y, 0) @ _rot("z", angles
↪ [1])
            @ _rot("x", angles[0]) @ _translate(-px, -SHOULDER_Y, 0))
```



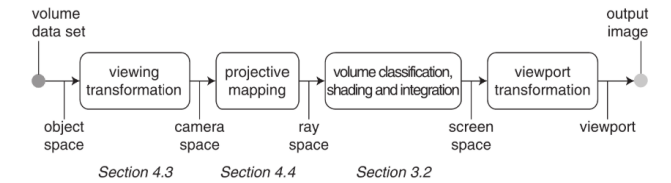
Hình 10. Cây phân cấp của nhân vật. Mỗi mũi tên là một phép nhân ma trận.

3. T2: Camera và phép chiếu

Một gaussian đi qua bốn bước để thành pixel trên màn hình [4]. Mỗi bước là một ô trong Hình 11, đọc từ trái sang phải.

- **Phần 1, phép biến đổi nhìn** (*viewing transformation*). Đưa gaussian từ không gian vật thể về không gian camera.
- **Phần 2, phép chiếu** (*projective mapping*). Biến ellipsoid 3D thành ellipse 2D.
- **Phần 3, tô màu và trộn** (*shading and integration*). Tính độ đục của từng pixel và trộn các gaussian với nhau.
- **Phần 4, phép biến đổi vùng nhìn** (*viewport transformation*). Đặt kết quả vào vùng nhìn của cửa sổ.

Năm phần dưới đây đi theo đúng thứ tự này.



Hình 11. Pipeline splatting của EWA [4]. Phép biến đổi nhìn là $\mathbf{W}$ và phép chiếu được xấp xỉ bằng $\mathbf{J}$.

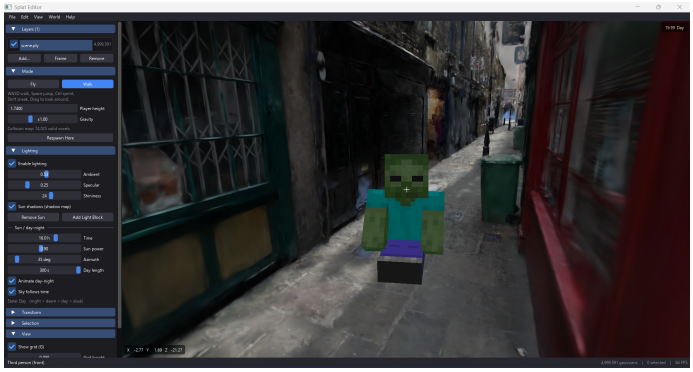
3.1. Phần 1: Phép biến đổi nhìn

Bước này đổi hệ tọa độ. Sau bước này, camera nằm ở gốc tọa độ và nhìn dọc theo trục $-z$. Hàm `look_at` tạo ba trục camera từ vị trí mắt $\mathbf{e}$ và điểm nhìn: phải $\mathbf{r}$, lên $\mathbf{u}$ và trước $\mathbf{f}$. Ma trận nhìn là

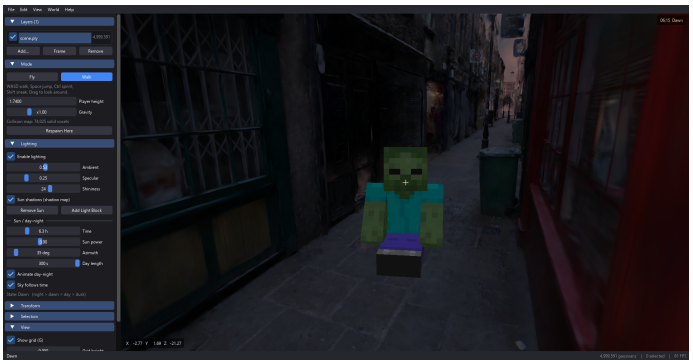
$$\mathbf{V} = \begin{pmatrix} r_x & r_y & r_z & -\mathbf{r}\cdot\mathbf{e} \\ u_x & u_y & u_z & -\mathbf{u}\cdot\mathbf{e} \\ -f_x & -f_y & -f_z & \mathbf{f}\cdot\mathbf{e} \\ 0 & 0 & 0 & 1 \end{pmatrix}. \tag{5}$$

Tâm gaussian được nhân với $\mathbf{V}$. Kết quả là tâm trong không gian camera (v_x, v_y, v_z). Hình dạng của gaussian chỉ bị quay. Ma trận $\mathbf{W}$ là phần quay của $\mathbf{V}$. Đó là khối 3×3 trên trái, trong ô vuông đỏ ở (5). Các hàng của nó là ba trục camera:

$$\mathbf{W} = \begin{pmatrix} r_x & r_y & r_z \\ u_x & u_y & u_z \\ -f_x & -f_y & -f_z \end{pmatrix}. \tag{6}$$



(a) Ban ngày, 15:59.



(b) Bình minh, 06:15.



(c) Môi trường 0,08 và một đèn.



(d) Hoàng hôn, 17:40.

Hình 9. Cảnh có mặt trời, một đèn điểm và nhân vật vào các giờ khác nhau trong ngày.

Ellipsoid (1) trong không gian camera là $\mathbf{W}\boldsymbol{\Sigma}\mathbf{W}^{\top}$.

Mã 6. `core/math3d.py`, `look_at`: ma trận nhìn $\mathbf{V}$

```
def look_at(eye, target, up):
    forward = target - eye
    forward /= np.linalg.norm(forward)
    right = np.cross(forward, up)
    right /= np.linalg.norm(right)
    true_up = np.cross(right, forward)
    m = np.identity(4, np.float32)
    m[0, :3] = right
    m[1, :3] = true_up
    m[2, :3] = -forward
    m[:3, 3] = -m[:3, :3] @ eye
    return m
```

3.2. Phần 2: Phép chiếu

Bước này đưa không gian camera lên mặt phẳng ảnh.

Chiếu một điểm. Ma trận chiếu phối cảnh dùng góc nhìn dọc φ ($k = 1/\tan(\varphi/2)$), tỉ lệ khung hình a, mặt phẳng gần n và xa f:

$$\mathbf{P} = \begin{pmatrix} \frac{k}{a} & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}. \tag{7}$$

Điểm được nhân với $\mathbf{P}$ rồi chia cho c_w. Kết quả là tọa độ chuẩn hóa (n_x, n_y) trong khoảng $[-1, 1]$. Phép chia cho c_w làm vật ở xa trông nhỏ hơn. Tâm gaussian được chiếu theo cách này.

Mã 7. `core/math3d.py`, `perspective`: ma trận chiếu $\mathbf{P}$

```
def perspective(fov_y_deg, aspect, near, far):
    zoom = 1.0 / math.tan(math.radians(fov_y_deg) / 2)
    m = np.zeros((4, 4), np.float32)
    m[0, 0] = zoom / aspect
    m[1, 1] = zoom
    m[2, 2] = (far + near) / (near - far)
    m[2, 3] = 2 * far * near / (near - far)
    m[3, 2] = -1
    return m
```

Chiếu một gaussian. Gaussian không phải là một điểm. Ellipsoid của nó phải thành một ellipse trên màn hình. Phép chia cho c_w không tuyến tính, nên không thể nhân $\mathbf{P}$ với ellipsoid. EWA thay phép chiếu bằng đạo hàm của nó (Jacobian) tại tâm gaussian:

$$\mathbf{J} = \begin{pmatrix} \frac{f_{px}}{-v_z} & 0 & \frac{f_{px}\, v_x}{v_z^2} \\ 0 & \frac{f_{px}}{-v_z} & \frac{f_{px}\, v_y}{v_z^2} \\ 0 & 0 & 0 \end{pmatrix}. \tag{8}$$

Ở đây $f_{px} = kH/2$ là tiêu cự tính bằng pixel, với k lấy từ (7) và H là chiều cao vùng nhìn. Ghép bước 1 và bước 2, hiệp phương sai 2D là

$$\boldsymbol{\Sigma}' = \mathbf{J}\mathbf{W}\boldsymbol{\Sigma}\mathbf{W}^{\top}\mathbf{J}^{\top}. \tag{9}$$

Ở đây $\boldsymbol{\Sigma}$ là hiệp phương sai 3D (1). $\mathbf{W}$ là phần quay của ma trận nhìn ở bước 1. $\mathbf{J}$ là Jacobian (8). Gọi khối 2×2 trên trái của $\boldsymbol{\Sigma}'$ là $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$. Giá trị riêng của nó là

$$\lambda_{1,2} = \frac{a+c}{2} \pm \sqrt{\left(\frac{a-c}{2}\right)^2 + b^2}. \tag{10}$$

Hai bán trục của ellipse tỉ lệ với $\sqrt{\lambda_1}$ và $\sqrt{\lambda_2}$. Shader vẽ mỗi gaussian thành một hình vuông phủ tối đa 3 độ lệch chuẩn. Nó bỏ các gaussian có tâm ở sau camera hoặc ngoài màn hình quá 1,3 lần.

Mã 8. render/shaders/splat_prep.vert: Jacobian $\mathbf{J}$ và $\Sigma' = \mathbf{J}\mathbf{W}\Sigma\mathbf{W}^{\top}\mathbf{J}^{\top}$

```
float vx = viewPos.x, vy = viewPos.y, vz = viewPos.z;
mat3 J = mat3(uFocal / -vz, 0.0, 0.0,
              0.0, uFocal / -vz, 0.0,
              uFocal * vx / (vz * vz), uFocal * vy / (vz * vz), 0.0);
mat3 T = J * mat3(uModelView) * linear;
mat3 cov = T * sigma * transpose(T);
float a = cov[0][0] + 0.3;
float b = cov[0][1];
float c = cov[1][1] + 0.3;
```

Mã 9. render/shaders/splat_prep.vert: giá trị riêng và hai trục ellipse

```
float mid = 0.5 * (a + c);
float radius = length(vec2(0.5 * (a - c), b));
float lambda1 = mid + radius;
float lambda2 = max(mid - radius, 0.1);
vec2 axis1 = (abs(b) < 1e-6) ? (a >= c ? vec2(1.0, 0.0) : vec2(0.0,
↪ 1.0))
                             : normalize(vec2(b, lambda1 - a));
vec2 axis2 = vec2(-axis1.y, axis1.x);
vec2 major = min(extent * sqrt(lambda1), 1024.0) * axis1 * toNdc;
vec2 minor = min(extent * sqrt(lambda2), 1024.0) * axis2 * toNdc;
```

3.3. Phần 3: Tô màu và trộn alpha

Bước này cho mỗi pixel một màu. Fragment shader xét từng pixel $\mathbf{x}$ của hình vuông. Nó tính hàm Gaussian 2D tại $\mathbf{x}$. Tâm $\boldsymbol{\mu}'$ là tâm gaussian đã chiếu. Hiệp phương sai $\boldsymbol{\Sigma}'$ lấy từ (9):

$$G(\mathbf{x}) = \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu}')^{\top}\,\boldsymbol{\Sigma}'^{-1}\,(\mathbf{x}-\boldsymbol{\mu}')\Big). \tag{11}$$

Số mũ chính là $-r^2/2$. Ở đây r là khoảng cách đến tâm ellipse, đo bằng độ lệch chuẩn. Độ đục của gaussian thứ i tại pixel là $a_i = \alpha_i\, G_i(\mathbf{x})$.

Mã 10. render/shaders/splat.frag: $a = \alpha\, e^{-r^2/2}$

```
float r2 = dot(vCorner, vCorner);
if (r2 > 9.0) discard;
float alpha = vColor.a * exp(-0.5 * r2);
if (alpha < 1.0 / 255.0) discard;
fragColor = vec4(vColor.rgb * alpha, alpha);
```

Các gaussian được vẽ từ xa đến gần và trộn với nhau. Màu pixel là

$$\mathbf{C} = \sum_i \mathbf{c}_i\, a_i\, T_i, \qquad T_i = \prod_{j<i}(1-a_j), \tag{12}$$

Ở đây các gaussian được đánh số từ gần đến xa. T_i là độ truyền (*transmittance*). Nó là phần ánh sáng còn lại sau khi đi qua các gaussian gần hơn i.

Chương trình vẽ từ xa đến gần. Nó cho cùng kết quả bằng phép trộn $\mathbf{C} \leftarrow \mathbf{c}\,a + (1-a)\,\mathbf{C}$ cho từng splat. Ở đây $\mathbf{C}$ là màu đã tích lũy.

Kết quả phụ thuộc thứ tự. Vì vậy phải sắp xếp theo độ sâu. Chương trình đổi độ sâu sang 16 bit. Nó sắp xếp bằng NumPy trong một luồng nền. Khi chưa sắp xếp xong, nó vẽ tạm bằng thứ tự của lần trước. Depth test được tắt. Lý do là gaussian trong suốt không được che mất gaussian phía sau.

Mã 11. render/splat_renderer.py: phép trộn $\mathbf{C} \leftarrow \mathbf{c}a + (1-a)\mathbf{C}$

```
GL.glDisable(GL.GL_DEPTH_TEST)
GL.glEnable(GL.GL_BLEND)
GL.glBlendFunc(GL.GL_ONE, GL.GL_ONE_MINUS_SRC_ALPHA)
GL.glDrawArraysInstanced(GL.GL_TRIANGLE_STRIP, 0, 4, self.draw_count)
```

Mã 12. render/sorter.py, DepthSorter.sort: sắp xếp xa → gần

```
depth[start:start + count] = positions[start:start + count] @ row[:3]
↪  + row[3]
visible = np.flatnonzero(depth < 0)
far_z, near_z = float(depth.min()), float(depth.max())
scale = 65535.0 / (near_z - far_z) if near_z > far_z else 0.0
depth_key = ((depth - far_z) * scale).astype(np.uint16)
return visible[np.argsort(depth_key, kind='stable')].astype(np.uint32
↪ )
```

3.4. Phần 4: Phép biến đổi vùng nhìn

Bước cuối đổi tọa độ chuẩn hóa (n_x, n_y) của bước 2 sang pixel. Vùng nhìn có kích thước $W \times H$ và góc (x_0, y_0):

$$x_{px} = x_0 + \tfrac{n_x+1}{2}W, \qquad y_{px} = y_0 + \tfrac{1-n_y}{2}H. \tag{13}$$

OpenGL tự làm bước này khi vẽ. Chương trình cũng dùng công thức này để biết một gaussian nằm ở pixel nào, ví dụ khi chọn bằng cọ (Mục 7).

Mã 13. editor/brush.py, BrushTool._to_screen: từ clip sang pixel

```
def _to_screen(clip, viewport):
    left, top, width, height = viewport
    x = left + (clip[..., 0] / clip[..., 3] + 1) * 0.5 * width
    y = top + (1 - clip[..., 1] / clip[..., 3]) * 0.5 * height
    return x, y
```

3.5. Phần 5: Ba loại camera

Camera quỹ đạo. Camera luôn nhìn vào điểm $\mathbf{t}$. Vị trí của nó dùng góc yaw θ, góc pitch ϕ và khoảng cách d:

$$\mathbf{e} = \mathbf{t} + d\,(\cos\phi\sin\theta,\ \sin\phi,\ \cos\phi\cos\theta). \tag{14}$$

Pitch bị giới hạn ở khoảng $\pm 89°$ để camera không lật.

Mã 14. render/camera.py, OrbitCamera.position

```
PITCH_LIMIT = 1.55
cp = math.cos(self.pitch)
offset = np.array([cp * math.sin(self.yaw), math.sin(self.pitch),
                   cp * math.cos(self.yaw)])
return self.target + self.distance * offset
```

Camera bay. Vận tốc không đổi ngay. Nó tiến dần đến vận tốc mong muốn $\mathbf{v}^*$, nên camera có quán tính:

$$\mathbf{v} \leftarrow \mathbf{v} + (\mathbf{v}^* - \mathbf{v})\min(10\Delta t, 1), \qquad \mathbf{e} \leftarrow \mathbf{e} + \mathbf{v}\,\Delta t. \tag{15}$$

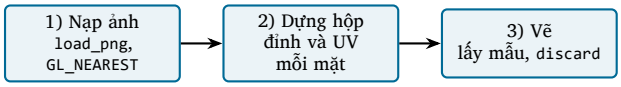
Mã 15. render/camera.py, OrbitCamera.fly

```
wanted = direction * self.fly_speed * (self.SPRINT_FACTOR if sprint
↪ else 1.0)
self.velocity += (wanted - self.velocity) * min(self.FLY_ACCEL * dt,
↪ 1.0)
self.target += self.velocity * dt
```

Góc nhìn thứ ba. Ở chế độ đi bộ, camera lùi ra sau nhân vật một đoạn. Chương trình bắn một tia từ mắt nhân vật đến vị trí camera trong bản đồ voxel. Nếu tia chạm tường, camera co lại ngay để không xuyên tường. Khi hết vướng, nó lùi ra dần.

4. T3: Ánh xạ texture

Chương trình dùng texture để phủ ảnh skin lên mô hình hộp của nhân vật. Nhân vật gồm 6 hộp (đầu, thân, hai tay, hai chân). Skin là một ảnh 64×64 (Hình 14) chứa hình trải phẳng của cả 6 hộp theo kiểu chữ thập Minecraft. Mỗi mặt hộp chỉ cần lấy đúng ô của nó trên ảnh. Quy trình có ba bước (Hình 12).



Hình 12. Ba bước vẽ skin.

1) Dựng hộp. Hộp có tâm $\mathbf{c}$, kích thước (w, h, d) và độ phồng δ. Mỗi góc của mặt là $\mathbf{p} = \mathbf{c} + \big(s_x(\frac{w}{2}+\delta),\, s_y(\frac{h}{2}+\delta),\, s_z(\frac{d}{2}+\delta)\big)$, $s_x, s_y, s_z \in \{\pm 1\}$. Hộp được dựng hai lần: lớp nền ($\delta = 0$) và lớp áo ($\delta = 0{,}25$ cho thân, tay, chân; 0,5 cho mũ). Lớp áo to hơn một chút nên không bị dính vào lớp nền.

Mã 16. `render/avatar_renderer.py`, `_box`: góc $\mathbf{p}$ của mặt

```
(cx, cy, cz), (w, h, d) = centre, size
hx, hy, hz = w / 2 + inflate, h / 2 + inflate, d / 2 + inflate
...
for (sx, sy, sz), (u, v) in zip(_FACES[face], uvs):
    vertices.append((cx + sx * hx, cy + sy * hy, cz + sz * hz, ...))
```

2) Chọn ô trên ảnh. Hộp có gốc (o_x, o_y) trên skin. Hình 13 cho thấy sáu mặt nằm ở đâu. Ví dụ đầu có gốc $(0, 0)$ và $w = h = d = 8$, nên mặt trước (khuôn mặt) nằm ở ô $(8, 8, 8, 8)$. Bốn góc của ô (r_x, r_y, r_w, r_h) nhận UV

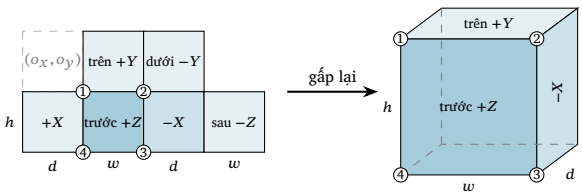
$$(u, v) = \Big(\frac{r_x + k\, r_w}{64},\ \frac{r_y + l\, r_h}{64}\Big), \quad (k, l) \in \{(0,0), (1,0), (1,1), (0,1)\}. \tag{16}$$

Thứ tự bốn góc phải khớp với thứ tự bốn đỉnh của mặt, nếu lệch thì ảnh bị lật hoặc xoay.

Mã 17. `render/avatar_renderer.py`: ô skin của mặt và UV

```
def _uv_region(face, ox, oy, w, h, d):
    return ((ox + d, oy + d, w, h), (ox + 2 * d + w, oy + d, w, h),
            (ox, oy + d, d, h), (ox + d + w, oy + d, d, h),
            (ox + d, oy, w, d), (ox + d + w, oy, w, d))[face]

rx, ry, rw, rh = _uv_region(face, ox, oy, w, h, d)
uvs = ((rx, ry), (rx + rw, ry), (rx + rw, ry + rh), (rx, ry + rh))
...  u / SKIN_SIZE, v / SKIN_SIZE
```



Hình 13. Hộp 3D và hình chữ thập của nó. Mặt $+Z$ ($w \times h$) có ô $(o_x{+}d,\, o_y{+}d,\, w,\, h)$, mặt $+Y$ có ô $(o_x{+}d,\, o_y,\, w,\, d)$. Bốn góc 1, 2, 3, 4 của mỗi mặt nhận $(k, l) = (0,0), (1,0), (1,1), (0,1)$ trong (16).

3) Vẽ. Skin được nạp bằng hàm `load_png` tự viết và lọc `GL_NEAREST`, nên pixel giữ nguyên độ sắc nét ($\lfloor 64u \rfloor, \lfloor 64v \rfloor$). Fragment shader bỏ pixel có $\alpha < 0{,}5$, nhờ đó chỗ trong suốt của lớp áo để lộ lớp nền. Màu cuối bằng màu texel nhân $0{,}62 + 0{,}38 \max(\mathbf{n} \cdot \mathbf{l}, 0)$. Mỗi hộp được vẽ với ma trận model riêng, nên tay và chân xoay độc lập quanh khớp.



Hình 14. Skin 64×64 của nhân vật. Mỗi hộp (đầu, thân, tay, chân) có một hình chữ thập cho lớp nền và một hình cho lớp áo (mũ, áo, tay áo).

5. T4: Ánh sáng và bóng đổ

Màu cuối của một gaussian được tạo qua bốn phần.

- **Phần 1, màu phụ thuộc hướng nhìn.** Đọc màu gốc từ các hệ số SH.
- **Phần 2, chiếu sáng Phong.** Cộng ánh sáng môi trường, khuếch tán và phản chiếu.
- **Phần 3, mặt trời.** Di chuyển mặt trời và màu trời theo giờ.
- **Phần 4, bóng đổ.** Kiểm tra từng gaussian với shadow map.

5.1. Phần 1: Màu phụ thuộc hướng nhìn

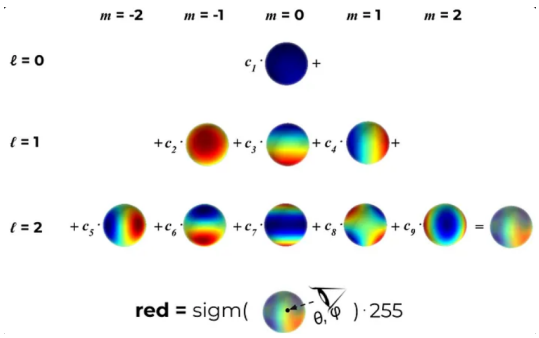
Màu của một gaussian thay đổi theo hướng nhìn $\mathbf{d}$. 3DGS lưu điều này bằng các hệ số điều hòa cầu (SH) [2]:

$$\mathbf{c}(\mathbf{d}) = \mathrm{clip}\Big(\mathbf{c}_0 + \sum_{k=1}^{K} Y_k(\mathbf{d})\, \mathbf{h}_k,\ 0,\ 1\Big). \tag{17}$$

Ở đây $\mathbf{c}_0$ là màu cơ bản, $\mathbf{h}_k$ là các hệ số trong file và Y_k là các hàm cơ sở. Các hệ số nằm trong không gian file. Vì vậy shader quay hướng nhìn về không gian đó: $\mathbf{d} = \mathrm{norm}(\mathbf{A}^{\top}(\mathbf{p}_w - \mathbf{e}))$. Nó không phải quay các hệ số trong mỗi khung hình. Hình 15 minh họa ý tưởng cho bậc 0 đến 2. Mỗi hàm cơ sở là một mẫu hình trên mặt cầu. Tổng có trọng số cho một màu ứng với mỗi hướng nhìn.

Mã 18. `render/shaders/splat_prep.vert`: màu SH theo hướng nhìn

```
vec3 c = -0.4886025119029199 * y * shCoeff(splat, 0)
       +  0.4886025119029199 * z * shCoeff(splat, 1)
       -  0.4886025119029199 * x * shCoeff(splat, 2);
...
vec3 color = vec3(rg, ba.x);
if (uShDegree > 0)
    color += shColor(splat, normalize(transpose(linear)
                                      * (center - uCamPos)));
color = clamp(color, 0.0, 1.0);
```



Hình 15. Điều hòa cầu đến bậc 2. Chín hàm cơ sở được nhân với các hệ số $c_1..c_9$ rồi cộng lại. Màu được đọc tại hướng nhìn (θ, φ).

5.2. Phần 2: Chiếu sáng Phong

Ánh sáng được tính một lần cho mỗi gaussian. Pháp tuyến $\mathbf{n}$ là trục ngắn nhất của ellipsoid, lật về phía camera. Màu cuối dùng mô hình Phong [5]. Mô hình có ba phần (Hình 16). Gọi $\mathbf{c}$ là màu của gaussian.

Ambient. Ánh sáng môi trường $\mathbf{L}_a$ chiếu đều lên mọi gaussian:

$$\mathbf{I}_a = \mathbf{c}\, \mathbf{L}_a. \tag{18}$$

Diffuse. Mặt hướng về đèn thì sáng hơn. Với đèn j, $\mathbf{L}_j$ là

màu và $\mathbf{l}_j$ là hướng về đèn:

$$\mathbf{I}_d = \mathbf{c} \sum_j \mathbf{L}_j \, w(\mathbf{n}, \mathbf{l}_j) \, S_j, \qquad w(\mathbf{n}, \mathbf{l}) = \mathrm{clip}\Big(\frac{\mathbf{n}\cdot\mathbf{l} + 0{,}35}{1{,}35}, 0, 1 \Big). \tag{19}$$

w là $\mathbf{n}\cdot\mathbf{l}$ đã làm mềm, vì pháp tuyến ước lượng khá nhiễu.

Specular. Điểm sáng bóng hiện ra khi tia phản xạ $\mathbf{r}_j$ gần với hướng $\mathbf{v}$ về camera:

$$\mathbf{I}_s = k_s \sum_j \mathbf{L}_j \, (\mathbf{r}_j \cdot \mathbf{v})_+^m \, S_j. \tag{20}$$

Ở đây k_s là cường độ specular và m là độ bóng. Phần này không dùng màu $\mathbf{c}$. Nó bằng 0 khi $\mathbf{n}\cdot\mathbf{l}_j \le 0$.

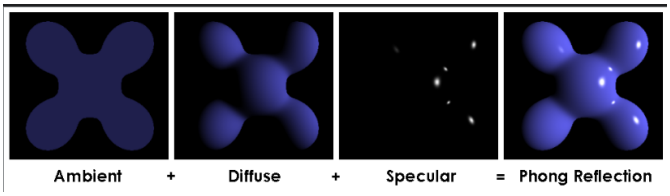
Tổng. Màu cuối là tổng của ba phần:

$$\mathbf{c}_{out} = \mathbf{I}_a + \mathbf{I}_d + \mathbf{I}_s. \tag{21}$$

S_j là hệ số bóng đổ của mặt trời, hoặc độ suy giảm theo khoảng cách của đèn điểm. Cảnh có một mặt trời và tối đa 8 đèn điểm. Đèn điểm không đổ bóng.

Mã 19. render/shaders/splat_prep.vert, shade: Phong

```
vec3 diffuse = uAmbient;
vec3 spec = vec3(0.0);
float ndl = dot(n, uSunDir);
float shadow = shadowFactor(worldPos, n);
diffuse += uSunColor * shadow
         * clamp((ndl + 0.35) / 1.35, 0.0, 1.0);
if (ndl > 0.0)
    spec += uSunColor * shadow * pow(max(dot(
        reflect(-uSunDir, n), v), 0.0), uShininess);
...
return albedo * diffuse + spec * uSpecular;
```



Hình 16. Mô hình Phong [5]: phần ambient, diffuse và specular, và tổng của chúng.

5.3. Phần 3: Mặt trời

Hướng mặt trời tính từ giờ h trong ngày. Mặt trời mọc lúc 6 giờ, cao nhất lúc 12 giờ và lặn lúc 18 giờ. Màu trời và ánh sáng môi trường đổi dần giữa ngày và đêm. Chúng hơi cam lúc hoàng hôn (Hình 9).

5.4. Phần 4: Bóng đổ

Chương trình vẽ tâm các gaussian đặc thành điểm vào một depth map 2048^2 nhìn từ mặt trời [6]. Mỗi gaussian được chiếu vào map này để so độ sâu. Nếu có vật gần mặt trời hơn, gaussian nằm trong bóng. Chương trình lấy trung bình 3×3 mẫu để làm mềm cạnh bóng (PCF [7]):

$$S = \frac{1}{9} \sum_{i,j=-1}^{1} \big[\, z_s - b \le D_s(u_s + i\tau, \, v_s + j\tau) \, \big]. \tag{22}$$

Ở đây (u_s, v_s, z_s) là tọa độ của gaussian trong map, D_s là độ sâu đã lưu, τ là bước texel và b là bias. Bias ngăn gaussian tự đổ bóng lên chính nó.

Mã 20. render/shaders/splat_prep.vert, shadowFactor: PCF 3×3

```
vec4 lp = uLightVP * vec4(worldPos + n * uShadowTexel * 2.0, 1.0);
vec3 s = lp.xyz / lp.w * 0.5 + 0.5;
float lit = 0.0;
for (int i = -1; i <= 1; i++)
    for (int j = -1; j <= 1; j++) {
        vec2 uv = s.xy + vec2(i, j) * uShadowInv;
        float d = textureLod(uShadowMap, uv, 0.0).r;
        lit += (s.z - uShadowBias <= d) ? 1.0 : 0.0;
    }
return lit / 9.0;
```

Mã 21. core/lighting.py, Lighting.shadow_setup: ma trận nhìn từ mặt trời

```
d = self.sun_direction()
r = self.radius * 1.1
eye = self.center + d * 2.0 * r
view = look_at(eye, self.center, up)
proj = orthographic(r, 0.01 * r, 4.0 * r)
return (proj @ view).astype(np.float32), texel, bias
```

6. T5: Hoạt họa và mô phỏng

Mọi chuyển động đều nhân với thời gian khung hình Δt. Nhờ vậy tốc độ ít phụ thuộc vào máy. Có bốn phần, theo thứ tự trong một khung hình.

- **Phần 1, chu kỳ ngày–đêm.** Chạy đồng hồ và di chuyển mặt trời.
- **Phần 2, va chạm voxel.** Biến các gaussian thành voxel đặc.
- **Phần 3, người chơi.** Đi, nhảy và rơi trên các voxel.
- **Phần 4, hoạt họa nhân vật.** Tính góc các khớp của nhân vật.

6.1. Phần 1: Chu kỳ ngày–đêm

Đây là một máy trạng thái có 4 trạng thái: NIGHT → DAWN → DAY → DUSK. Đồng hồ tự chạy. Đèn đặt "bật vào ban đêm" sẽ bật lúc hoàng hôn và tắt lúc bình minh. Các trạng thái bắt đầu lúc 5 giờ (DAWN), 7 giờ (DAY), 17 giờ (DUSK) và 19 giờ (NIGHT). Một ngày dài $T_{day} = 36$ giây, nên mỗi khung hình giờ tăng $\Delta h = 24\,\Delta t / T_{day}$. Vị trí mặt trời phụ thuộc vào giờ:

$$\theta = \frac{h - 6}{12}\pi, \qquad \epsilon = 65^\circ \sin\theta, \quad \mathbf{s} = \big(\cos\epsilon\cos\phi, \; \sin\epsilon, \; \cos\epsilon\sin\phi \big), \tag{23}$$

trong đó $\phi = \phi_0 + \theta - \pi/2$ và ϕ_0 là góc phương vị của mặt trời. Mặt trời mọc lúc 6 giờ, cao nhất lúc 12 giờ và lặn lúc 18 giờ. Hệ số ánh sáng ngày là $D = \mathrm{smoothstep}(-0{,}08, \, 0{,}3, \, s_y)$. Màu môi trường và màu bầu trời được trộn giữa giá trị ban đêm và ban ngày theo D, bầu trời có thêm sắc cam gần đường chân trời.

Mã 22. core/lighting.py: giờ, hướng mặt trời và hệ số ánh sáng ngày

```
hours_per_second = 24.0 / max(self.day_length, 1.0)
self.hour = (self.hour + dt * hours_per_second) % 24.0

angle = (self.hour - 6.0) / 12.0 * math.pi
elevation = math.sin(angle) * math.radians(MAX_SUN_ELEVATION)
azimuth = (math.radians(self.sun_azimuth)
           + (angle - math.pi / 2))
return np.array([math.cos(elevation) * math.cos(azimuth),
                 math.sin(elevation),
                 math.cos(elevation) * math.sin(azimuth)])

return _smooth(-0.08, 0.3, self.sun_direction()[1])
```

6.2. Phần 2: Va chạm với voxel

Gaussian không có bề mặt rõ ràng, nên khó tính va chạm trực tiếp. Chương trình chia không gian thành các voxel cạnh v. Voxel có đủ gaussian là voxel đặc. Mỗi voxel đặc được lưu

thành một số nguyên 64 bit:

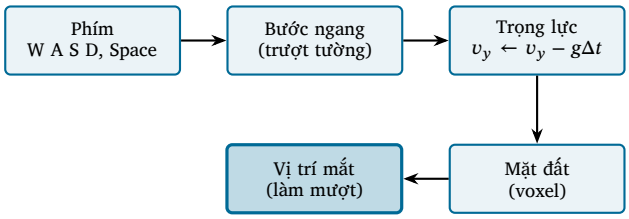
$$\mathbf{c} = \left\lfloor \frac{\mathbf{p}-\mathbf{o}}{v} \right\rfloor, \qquad \text{key} = (c_x \ll 42) \,|\, (c_z \ll 21) \,|\, c_y. \tag{24}$$

Các khóa được sắp xếp. Vì c_y nằm ở các bit thấp, các ô cùng một cột đứng liền nhau. Tìm mặt đất dưới chân chỉ cần một lệnh `np.searchsorted`.

Mã 23. `core/physics.py`, `CollisionWorld`: khóa voxel và tìm mặt đất

```
cells = np.floor((positions - self.origin) / self.voxel).astype(np.
↪ int64)
keys = (cells[:, 0] << (2 * _BITS)) | (cells[:, 2] << _BITS) | cells
↪ [:, 1]
self.keys = np.unique(keys)

idx = np.searchsorted(self.keys, query, side='right') - 1
```



Hình 17. Một khung hình của chế độ đi bộ.

6.3. Phần 3: Người chơi

Hình 17 tóm tắt một khung hình. Người chơi là một hình trụ. Khi gặp tường, người chơi trượt dọc theo tường. Người chơi leo được bậc thấp và nhảy được.

Trọng lực. Khi không đứng trên đất, chương trình tích phân theo Euler:

$$v_y \leftarrow v_y - g\,\Delta t, \qquad y \leftarrow y + v_y\,\Delta t. \tag{25}$$

Tỉ lệ và cú nhảy. Mọi kích thước và tốc độ tỉ lệ theo chiều cao người chơi H. Lý do là cảnh 3DGS không có đơn vị cố định. Các hằng số là: cạnh voxel $v = H/8$, bán kính người chơi $0{,}14H$, chiều cao bậc $0{,}35H$, trọng lực $g = 12H$, tốc độ đi $2{,}5H$ mỗi giây (chạy nhanh $\times 1{,}7$, đi chậm $\times 0{,}4$). Cú nhảy đạt độ cao $0{,}75H$, nên vận tốc lúc bật lên là

$$v_0 = \sqrt{2\,g\,(0{,}75H)}. \tag{26}$$

Làm mượt mắt. Độ cao mắt được làm mượt khi người chơi đứng trên đất, để bề mặt gaussian gồ ghề không làm rung camera.

Mã 24. `core/physics.py`, `Physics.walk`: nhảy và trọng lực

```
if player.on_ground and jump:
    player.vel_y = math.sqrt(2.0 * self.gravity * self.jump_ratio * h
↪ )
    player.on_ground = False
if not player.on_ground:
    player.vel_y -= self.gravity * dt
    new_y = y + player.vel_y * dt
```

6.4. Phần 4: Hoạt họa nhân vật

Chương trình chỉ tính góc các khớp (đơn vị độ); renderer dựng ma trận. Gọi d là quãng đường đi được trong khung hình này, tính theo chiều cao người. Pha của bước đi tăng theo quãng đường chứ không theo thời gian, nên chân không bị trượt trên đất:

$$\psi \leftarrow \psi + 1{,}75 \cdot 2\,d, \qquad \theta_{chn} = \pm A\,a\,\sin\psi, \quad \theta_{tay} = \mp A\,a\,\sin\psi, \tag{27}$$

với $A = 60°$. Chân và tay đối diện chuyển động cùng nhau.

Trộn trạng thái. Biên độ $a \in [0,1]$ bám theo tốc độ bằng bộ lọc $x \leftarrow x + (x^* - x)\min(1, k\,\Delta t)$ (15), với $k = 8$. Cùng bộ lọc này, với $k = 7$, trộn các trạng thái khác: ngồi (chân $-90°$, thân hạ xuống), cúi (thân nghiêng $28°$) và các cử chỉ vẫy tay, zombie, cổ vũ. Hai tư thế được trộn tuyến tính, $\theta = (1-w)\theta_a + w\,\theta_b$, nên cử chỉ hiện ra và mất đi mượt. Khi đứng yên, một số hạng sin nhỏ làm tay lắc nhẹ để giả lập nhịp thở.

Mã 25. `core/avatar_pose.py`: nhịp đi và bộ lọc làm mượt

```
def _approach(value, target, rate, dt):
    return value + (target - value) * min(1.0, dt * rate)

walked = distance / max(height, 1e-9)
self.amount = _approach(self.amount, target, BLEND, dt)
self.phase += walked * STRIDE * GAIT_SCALE
swing = math.sin(self.phase) * SWING_ANGLE * walk
leg_l, leg_r = swing, -swing
arm_l = [-swing + idle, 0.0]
arm_r = [swing - idle, 0.0]
```

Ma trận khớp. Mỗi chi là một phép quay quanh khớp của nó: $\mathbf{M} = \mathbf{T}(\mathbf{j})\,\mathbf{R}(\theta)\,\mathbf{T}(-\mathbf{j})$, nhân sau ma trận của khối cha, ví dụ tay nhân sau thân.

7. T6: Tương tác và giao diện

Tương tác có bốn phần, từ đầu vào đến màn hình.

- **Phần 1, xử lý sự kiện.** Quyết định ai nhận mỗi sự kiện chuột hoặc phím.
- **Phần 2, chọn bằng cọ.** Chọn gaussian bằng cách tô lên màn hình.
- **Phần 3, xóa mềm và hoàn tác.** Xóa vùng chọn và khôi phục.
- **Phần 4, giao diện.** Bảng điều khiển, HUD và phím tắt.

7.1. Phần 1: Xử lý sự kiện

Sự kiện chuột và bàn phím đến ImGui trước. Nếu ImGui không dùng, chúng đến camera, cọ chọn và các phím tắt.

7.2. Phần 2: Chọn bằng cọ

Nhấn `B` để bật cọ. Tô lên màn hình để chọn gaussian. Chúng chuyển sang màu vàng. Tâm mỗi gaussian được chiếu thành pixel $\mathbf{p}$ (Mục 3.1).

Kiểm tra capsule. Một nét vẽ từ $\mathbf{a}$ đến $\mathbf{b}$ là một capsule bán kính r. Gaussian được chọn nếu

$$\|\mathbf{p} - \mathbf{a} - \tau(\mathbf{b}-\mathbf{a})\| \le r, \quad \tau = \text{clip}\,\frac{(\mathbf{p}-\mathbf{a})\cdot(\mathbf{b}-\mathbf{a})}{\|\mathbf{b}-\mathbf{a}\|^2} \in [0,1]. \tag{28}$$

Tức là khoảng cách từ $\mathbf{p}$ đến đoạn $\mathbf{ab}$ không quá r.

Mã 26. `editor/brush.py`, `BrushTool._segment_dist2`: kiểm tra capsule

```
def _segment_dist2(p, a, b):
    ab = b - a
    length2 = float(ab @ ab)
    t = (np.clip((p - a) @ ab / length2, 0, 1) if length2 > 0
         else np.zeros(p.shape[:-1]))
    nearest = a + t[..., None] * ab
    diff = p - nearest
    return (diff * diff).sum(axis=-1)
```

Tính chất. Cọ chỉ xét tâm gaussian và không kiểm tra độ sâu. Vì vậy nó chọn xuyên qua cảnh, giống SuperSplat [3]. Để nhanh, cọ loại cả khối 1024 gaussian khi hộp bao của khối nằm ngoài nét vẽ.

7.3. Phần 3: Xóa mềm và hoàn tác

Mỗi gaussian có một byte trạng thái trên GPU với hai bit: *đã chọn* và *đã xóa*. Lệnh `Delete` chỉ bật bit "đã xóa". Shader bỏ qua các gaussian này. Dữ liệu gốc không đổi. Mỗi lần xóa là một lệnh trong `History` (mẫu Command [8], tối đa 20 bước).

`Ctrl+Z` khôi phục lệnh cuối.

Mã 27. `core/layer.py` và `core/history.py`: xóa mềm và hoàn tác

```
def delete_selected(self):
    indices = np.flatnonzero(self.selection)
    self.deleted[indices] = True
    self.selection[indices] = False
    return indices

def push(self, command):
    self._stack.append(command)
    del self._stack[:-self.max_steps]

def undo(self):
    command = self._stack.pop()
    command.undo()
```

7.4. Phần 4: Giao diện

Bảng ImGui bên trái điều khiển layer, phép biến đổi, vùng chọn, chế độ camera và ánh sáng. HUD hiển thị tâm ngắm, đồng hồ, vị trí và vòng tròn cọ. Bảng 3 liệt kê các phím chính.

Bảng 3. Các phím điều khiển chính.

Phím / chuột	Chức năng
Kéo chuột trái / giữa	Xoay quanh / pan
Kéo chuột phải + `WASD`	Nhìn quanh, bay (`Ctrl` = nhanh)
`M`, `F5`, `C`, `V`	Bay/đi bộ; góc nhìn thứ ba; ngồi; vẫy tay
`F`, `G`	Xem toàn cảnh; hiện/ẩn lưới
`B`, `[`, `]`	Cọ chọn; cỡ cọ
`Delete`, `Ctrl+Z`	Xóa / hoàn tác
`Ctrl+O/I/S`	Mở / thêm layer / lưu

8. Hạn chế và hướng phát triển

8.1. Hạn chế

- **Tốc độ phụ thuộc CPU.** Độ sâu được sắp xếp bằng NumPy trong một luồng nền (Mục 3.3). Khi camera quay nhanh, hình có thể nhấp nháy, và cảnh lớn phải giới hạn số gaussian vẽ (LOD).
- **Hình học xấp xỉ.** Gaussian không có bề mặt thật. Cọ chọn, va chạm, pháp tuyến và bóng đều dựa vào tâm gaussian, nên cọ chọn cả vật bị che, va chạm có thể thủng lỗ và ánh sáng còn thô.

8.2. Hướng phát triển

- **Hỗ trợ mô phỏng.** Dùng bản đồ voxel và phần vật lý có sẵn để mô phỏng chuyển động trong cảnh quay từ đời thực, ví dụ thử đường đi của người, robot hay xe. Cần va chạm chính xác hơn và thêm các vật thể động.
- **Ứng dụng vào game.** Biến cảnh 3DGS thành một màn chơi: thêm nhân vật máy, vật phẩm và nhiệm vụ, giữ chu kỳ ngày đêm và đèn. Cảnh đã làm sạch có thể xuất sang game engine như Unity hoặc Unreal.

9. Tuyên bố sử dụng AI

Theo quy định của môn học, sinh viên được dùng công cụ AI (ChatGPT, Gemini) để học khái niệm, gỡ lỗi và cải thiện mã nguồn. Trong đồ án này, tôi đã dùng ChatGPT và Gemini cho hai việc:

- **Học khái niệm.** Giải thích 3D Gaussian Splatting, EWA, điều hòa cầu, bản đồ bóng và các kỹ thuật đồ họa liên quan.
- **Gỡ lỗi và cải thiện mã.** Tìm lỗi, gợi ý cách viết gọn và nhanh hơn cho chương trình.

Tôi đã đọc, kiểm tra và chạy thử nội dung do AI gợi ý, và chịu trách nhiệm về toàn bộ mã nguồn và báo cáo này.

10. Kết luận

Splat Editor đọc, hiển thị, chỉnh sửa và lưu được file 3DGS. Chương trình dùng đủ sáu nhóm kỹ thuật của môn học. Bảng 4 chỉ vị trí của từng phần trong mã nguồn.

Bảng 4. Các file mã nguồn chính (trong `splat_editor/`).

Phần	File
Đọc/ghi PLY	`io/ply.py`, `io/codec.py`
Hiển thị gaussian	`render/splat_renderer.py`, `render/sorter.py`, `render/shaders/splat_prep.vert`
T1	`core/layer.py`, `core/transform.py`, `core/avatar_pose.py`
T2	`core/math3d.py`, `render/camera.py`
T3	`render/avatar_renderer.py`
T4	`core/sh.py`, `core/lighting.py`, `render/shaders/shadow.vert`
T5	`core/physics.py`, `core/avatar_pose.py`
T6	`editor/brush.py`, `core/history.py`, `ui/`

Tài liệu tham khảo

[1] J. L. Schönberger, J.-M. Frahm, Structure-from-motion revisited, in: CVPR, 2016, pp. 4104–4113.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis, 3D Gaussian splatting for real-time radiance field rendering, ACM Trans. Graph. 42(4) (2023) 139.

[3] PlayCanvas, SuperSplat: an open-source 3D Gaussian Splat editor, https://github.com/playcanvas/supersplat.

[4] M. Zwicker, H. Pfister, J. van Baar, M. Gross, EWA splatting, IEEE Trans. Vis. Comput. Graph. 8(3) (2002) 223–238.

[5] B. T. Phong, Illumination for computer generated pictures, Commun. ACM 18(6) (1975) 311–317.

[6] L. Williams, Casting curved shadows on curved surfaces, in: SIGGRAPH, 1978, pp. 270–274.

[7] W. T. Reeves, D. H. Salesin, R. L. Cook, Rendering antialiased shadows with depth maps, in: SIGGRAPH, 1987, pp. 283–291.

[8] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.